

Shader Spec Eval: Reference-Free Behavioral Evaluation of LLM-Generated Visual Programs

Husien Vora

Independent researcher

Pre-results manuscript, draft 0.2 — 24 August 2026

This draft describes the benchmark and a small local pilot. The frozen multi-model experiment has not yet been run. Every uncollected result is marked **TBD** and must not be cited as a finding.

Abstract

Evaluations of generated graphics code face an awkward choice. Source-level tests may reject an unfamiliar but valid implementation, image comparison may reject harmless pixel differences, and compilation alone accepts programs that render the wrong behavior. We introduce Shader Spec Eval, an experimental benchmark in which a language model translates a natural-language visual specification into a GLSL ES fragment shader. The shader is rendered in a deterministic headless browser and evaluated with task-specific measurements of direction, repetition, geometry, color, symmetry, temporal behavior, and stability. The grader uses neither a reference-image distance nor an LLM judge. The current frozen suite contains 15 tasks. Each grader is exercised with at least two independently structured valid shaders and targeted invalid controls. A preliminary five-task local pilot with Qwen2.5-Coder 14B compiled all 15 samples, achieved 87.84% mean property compliance, and produced 5/15 completely compliant shaders; this is evidence that the pipeline runs, not a comparative model result. We specify a preregistered temperature-controlled study of six hosted model routes and one local baseline, ten samples per task. The contribution is a reproducible method and open test bed for discussion, not a claim of a universal shader benchmark.

1. Introduction

Code-generation benchmarks commonly succeed because their outputs expose a conventional interface: a generated function can be called with test inputs and its return values checked. A shader's interface is visual and temporal. It maps coordinates, time, resolution, and other uniforms to pixels, and many different programs can implement the same requested effect.

This makes three simple grading strategies incomplete. Exact source matching has no meaningful notion of implementation equivalence. Reference-image comparison can punish antialiasing, precision, or stylistic differences that leave the requested behavior intact. Compilation checks syntax and API use but not whether an eight-by-eight grid became sixteen-by-sixteen, whether a circle is off-center, or whether motion travels in the wrong direction.

Shader Spec Eval treats rendered requirements as executable visual specifications. A prompt asks for measurable behavior. A model returns a complete shader. A controlled renderer samples the result at

specified resolutions and times, and property checkers answer narrow questions about the render. Partial property compliance and all-properties-clean completion are both reported.

The intended claim is modest: property-based rendering may be a useful way to evaluate behavioral compliance in generated visual programs. This paper documents the construction, validates the grader against authored controls, and defines the experiment required to learn whether the suite is reliable and discriminating across models.

2. Relationship to prior work

HumanEval established functional correctness through execution as a central measure for generated code [1]. ShaderEval more directly studies language models on graphics-code completion using functions derived from existing Shadertoy programs [2, 3]. Shader Spec Eval asks a different question: can a model produce a complete visual program from a behavioral natural-language specification, when any implementation satisfying the rendered properties is acceptable?

The design is also related to property-based testing, where general invariants and generated cases replace a small list of exact expected outputs [4]. Here, the sampled cases are render conditions— coordinates, times, and resolutions—and the invariants describe visible behavior. This analogy has limits: finite raster samples cannot prove shader semantics, and every image-derived threshold can produce false positives or negatives.

We do not claim to introduce the first shader evaluation, the first execution-based code benchmark, or a general test of spatial intelligence. The proposed novelty is the combination of behavioral natural-language shader synthesis, reference-free image-property graders, temporal sampling, and published valid/invalid grader controls.

3. Benchmark design

3.1 Task format

Each task contains a natural-language prompt and a JSON specification. The model must output one complete GLSL ES 3.0 fragment shader. Prompts define available uniforms and prohibit external textures. The model does not receive the reference shaders, rendered images, property thresholds, compiler output, or grader feedback in the primary condition.

The 15 tasks deliberately reuse concepts in changed positions and combinations. This reduces the chance that one generic visual heuristic stands in for the requested behavior.

Task	Principal requirements
gradient	horizontal direction, endpoint colors, time stability
gradient-diagonal	joint x/y direction rather than x alone
bands	exact repeated band count and orientation
stripes	exact stripe count and hard transitions
tile	exact 8×8 checker repetition and hard edges
grid	exact two-axis grid repetition

Task	Principal requirements
sdf	centered circular boundary, radius, and color
offset-circle	circular boundary at a specified noncentral point
ring	independently constrained inner and outer radii
box	unequal rectangular bounds rather than a generic square
double-circle	two objects at specified positions and sizes
mirror	bilateral structure with deliberately off-center content
pulse	centered radial falloff, symmetry, and nonperiodic motion sample
motion	object centroid moves in the requested direction
polar	exact six-fold angular order, color, and rotation

3.2 Rendering

Shaders are compiled and rendered through WebGL 2 in headless Chromium. The project records the source, compiler log, render images, resolution, sample times, elapsed time, model sampling configuration, and provider metadata. Human reference shaders are tested through the identical render-and-grade path; they are sanity controls and do not define a pixel target.

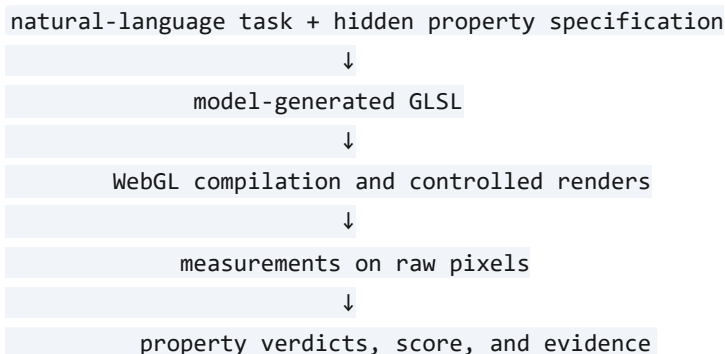
3.3 Property checkers

Checkers measure narrow observable claims including color-channel dominance, axis or diagonal change, sharpness, periodic counts, centroid location and movement, bilateral or rotational symmetry, circular and rectangular bounds, ring radii, change across time, and time stability. Each output receives a Boolean verdict and diagnostic values for every property. The property score is the fraction of checks passed. Clean completion requires every check on a task to pass.

Properties from one shader are correlated. Therefore property verdicts are useful diagnostics but must not be treated as independent statistical observations. Task/model/seed is the basic sampled unit for model comparisons.

3.4 Implementation mechanism

The evaluation pipeline can be summarized as:



Each task separates what the model sees from how the output is graded. `task.json` contains the public natural-language request. `shader_spec.json` contains the render resolution, fixed time samples, property names, expected values, and tolerances. The latter is not included in the model request. This separation prevents the model from targeting individual sample coordinates or grader thresholds directly.

For every model, task, and sampling seed, the runner creates one result cell. It sends a stateless request asking for shader source only and records the raw response, temperature, seed, token usage, provider-reported cost, resolved model, and provider. Markdown fences are removed when present, but the program is otherwise not rewritten. The primary study permits no compiler-repair attempt.

The renderer launches headless Chromium and creates a fresh WebGL 2 context. A generated Shadertoy-style `mainImage` function is surrounded with a small standard prelude and `main` function; a shader that already supplies a complete program is used directly. The harness compiles the fragment shader, draws a full-screen triangle, assigns fixed `iResolution` and `iTime` values, and reads the complete RGBA framebuffer back into Python. Animated and stability tasks render the same source at multiple predetermined time values. Wall-clock time is never exposed to the shader.

The property dispatcher reads the hidden JSON list and calls a matching image-measurement function. Single-frame checks inspect geometry, repetition, color, edges, or symmetry. Multi-frame checks measure animation, stability, or centroid displacement. Each function returns a Boolean verdict and a human-readable diagnostic such as the observed radius, number of bands, color difference, or movement delta. Unknown property names fail closed.

A cell's property score is the number of passed properties divided by the number evaluated. Clean completion requires all properties to pass. A shader compile failure is a genuine zero because no executable visual program was produced. Provider, transport, and renderer failures are missing observations rather than model-quality failures, so they are retained in the raw evidence but excluded from aggregates. Results are checkpointed after every cell and keyed by model, task, and seed for safe resumption.

3.5 Worked example: measuring a ring

The ring task asks for a centered blue ring with inner radius 0.18 and outer radius 0.30 of frame height. The hidden specification renders a 256×256 frame at three fixed times and requests checks for nonblank output, finite pixels, two ring radii, hard boundaries, center and ring colors, horizontal and vertical symmetry, blue dominance, and time stability.

The `ring_radii` checker first samples the expected middle of the ring at 32 angles and compares it with samples from the center and exterior. If there is insufficient foreground/background contrast, the check fails. Otherwise, a brightness threshold is placed between the estimated ring and background levels. The checker casts 32 rays from the image center, records where every ray enters and leaves the foreground, converts those pixel distances to normalized radii, and compares the extrema with 0.18 and 0.30 using a tolerance of 0.035.

Consequently, a filled circle fails because it lacks an inner transition; a ring of the wrong thickness fails its radii; and a square produces angle-dependent distances that exceed tolerance. The implementation is not prescribed: `step`, `smoothstep`, a signed-distance expression, or any other construction can pass when its rendered behavior meets the same measurements.

3.6 Meaning of reference-free

Human-authored shaders are present as grader-validation controls and as an optional dashboard row. They are not pixel targets. A candidate score never contains an image-distance term between the candidate and a reference render. Instead, the human shader, an alternate valid implementation, and every model output are

independently sent through the same property functions. “Reference-free” therefore describes candidate scoring, not the absence of human-authored validation artifacts.

This approach accepts implementation diversity but does not prove equivalence for all possible inputs. A finite set of pixel, resolution, and time samples cannot establish that an internal technique was used, guarantee performance, judge aesthetics, or rule out a shader constructed to exploit known samples. It evaluates sampled behavioral compliance under a controlled renderer.

4. Grader validation

The benchmark validates the test before testing models. Every frozen task has a canonical shader and a structurally different valid alternative. It also has targeted controls that are visually plausible while violating a named requirement. Examples include an x-only implementation of a diagonal gradient, four instead of the required number of bands, 8 instead of 16 stripes, a 12×8 grid, the wrong circle radius, an inner ring boundary in the wrong place, a square in place of a circle, centered content for an off-center mirror test, static motion, and motion in the opposite direction.

The self-test requires every valid implementation to pass all properties and each invalid control to fail the intended property. This does not establish complete validity: authored controls can only cover anticipated failure modes. It does establish that common near misses are not knowingly accepted by the frozen graders.

During benchmark development, controls exposed overly broad checks—for example, accepting the wrong repetition count or a symmetric shape with the wrong geometry. Those defects were corrected before freezing the study. After model outputs are inspected, threshold tuning is prohibited. A substantive grader fix creates a new version and requires affected outputs to be rerun.

5. Experimental protocol

5.1 Frozen primary study

The planned primary study uses the manifest `temperature-controlled-v0.2`. It contains 15 tasks, ten seeds per task, six OpenRouter model routes, and a local Qwen2.5-Coder 14B baseline, for 1,050 generated shaders (900 hosted and 150 local).

The hosted routes frozen on 24 August 2026 are Gemini 3.7 Flash, Grok 4.5, Qwen3 Coder, DeepSeek V3.2, GLM 4.7 Flash, and Kimi K2.7 Code. Their identifiers and current pricing are stored in the machine-readable manifest and captured again immediately before execution. A route is eligible only when OpenRouter's live catalog advertises support for both `temperature` and `seed`.

Temperature is fixed at 0.7 and seeds are 1 through 10. `top_p` is left unchanged. Nonzero temperature is a deliberate choice to measure sampling variability, not a claim that professional evaluations must always use nonzero temperature. The important requirement is that the policy is explicit, constant, and recorded. Provider fallback is disabled. Requests are stateless and tool free, and output is capped at 1,024 tokens.

The first response is the primary result. Syntax-repair retries are disabled for this study. Transport failures may be retried without changing the seed or prompt and are reported separately. Provider errors are not scored as model failures until a valid response is obtained or the route is declared unavailable according to a published rule.

5.2 Reproducibility and cost control

Before paid calls, the runner validates all known-good and adversarial grader controls and performs a live model-parameter preflight. It saves the catalog snapshot and exact run configuration, checkpoints each response, and resumes by model/task/seed key. Returned token counts, provider cost, resolved model, and provider name are retained. The catalog-price projection must remain below a \$5 operational ceiling. The projection is a safety bound rather than an expected study cost.

6. Measures and analysis

The study will report:

1. compile rate;
2. clean completion rate;
3. task-macro-averaged property compliance;
4. per-task and per-property outcomes;
5. failure categories, including provider and extraction failures;
6. within-model variation across seeds;
7. task-level 95% confidence intervals and paired model differences; and
8. input/output tokens, latency, and provider-reported cost.

Rankings will not be inferred from small mean differences whose task-level uncertainty overlaps. The study has only 15 task concepts, so tasks—not hundreds of correlated properties—limit the strength of generalization. Any exploratory analysis discovered after viewing results will be labeled exploratory.

7. Preliminary local pilot

An earlier engineering pilot used Qwen2.5-Coder 14B (Q4_K_M) at temperature 0.7 with seeds 7, 8, and 9 on the original five-task ladder. Across 15 generated shaders:

- 15/15 compiled;
- mean property compliance was 87.84%;
- 5/15 satisfied every property; and
- the standard deviation across three full-ladder repeat scores was 3.36 percentage points.

These numbers show that the pipeline can identify partial behavioral failures even when code compiles. They do not estimate comparative validity, performance on the expanded suite, or a stable population score.

8. Multi-model results

TBD — the frozen experiment has not been executed. Do not cite this section as a finding.

Model	Compile rate	Clean completion	Property compliance	95% CI	Cost
Gemini 3.7 Flash	TBD	TBD	TBD	TBD	TBD

Model	Compile rate	Clean completion	Property compliance	95% CI	Cost
Grok 4.5	TBD	TBD	TBD	TBD	TBD
Qwen3 Coder	TBD	TBD	TBD	TBD	TBD
DeepSeek V3.2	TBD	TBD	TBD	TBD	TBD
GLM 4.7 Flash	TBD	TBD	TBD	TBD	TBD
Kimi K2.7 Code	TBD	TBD	TBD	TBD	TBD
Qwen2.5-Coder 14B local	TBD	TBD	TBD	TBD	local

The final manuscript must add task-level uncertainty, paired differences, failure examples, and a discussion of whether observed score spread exceeds within-model variation.

9. Threats to validity

Construct validity is the central limitation. Success may reflect GLSL familiarity, instruction following, or coding ability as much as spatial reasoning. Rendered pixels cannot prove that a requested internal technique such as a signed-distance function was used. The current suite grades compliance, not aesthetics, creativity, performance, or maintainability.

Finite resolutions and times allow adversarial or accidental false positives. Thresholds encode author judgment. Headless Chromium and GPU/software-rendering versions can affect boundary pixels. The model sample is selected partly by cost and parameter support, and routed hosted identifiers may change over time. OpenRouter seed support does not guarantee bit-identical inference across hardware or providers. Disabling fallback and recording the resolved provider reduces, but does not eliminate, this risk.

Fifteen tasks are insufficient for broad claims about shader programming. Ten samples improve estimation of stochastic variability but do not create more task diversity. External validity therefore requires community-authored tasks, hidden controls, additional render environments, and replication by other researchers.

10. Open-science position

The benchmark, prompts, shaders, controls, thresholds, runner, and raw results are intended to be released under an open license. Public timestamps document the work but do not grant ownership of a research direction. The useful protection is transparent prior art: a versioned repository, archived release, dated video, and clearly scoped claims that let others reproduce, criticize, and extend the method with attribution.

11. Conclusion

Shader Spec Eval proposes a small, inspectable bridge between code execution and visual behavior. Its strongest current evidence concerns grader engineering: independently written valid shaders are accepted, targeted near misses are rejected, and a local pilot produces meaningful partial failures. Whether the benchmark reliably separates models remains an empirical question reserved for the frozen study. Releasing the unfinished method and its limitations is part of the contribution: it creates a concrete artifact around which a better visual-code evaluation can be built.

References

- [1] M. Chen et al. “Evaluating Large Language Models Trained on Code.” 2021. [arXiv:2107.03374](#).
- [2] “Evaluating Language Models for Computer Graphics Code Completion.” LLM4Code 2025. DOI: [10.1109/LLM4Code66737.2025.00017](#).
- [3] Vipitis. “ShaderEval inputs.” [Hugging Face dataset](#).
- [4] K. Claessen and J. Hughes. “QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs.” ICFP 2000. DOI: [10.1145/351240.351266](#).
- [5] OpenRouter. “Models and supported parameters” and “Models API.” Accessed 24 August 2026. [Model catalog](#); [API reference](#).
- [6] OpenAI. “Create a model response: temperature.” Accessed 24 August 2026. [API reference](#).